

Improving Crop Disease Detection Using Multi-Dataset Training and Domain Generalization

Deliverable 2: Model Architecture, Implementation, Training, and Evaluation

Ubaid

Department of Computer Science

University Name

April 22, 2026

Abstract

This deliverable presents the model architecture, implementation, training strategy, and evaluation results for the project titled *Improving Crop Disease Detection Using Multi-Dataset Training and Domain Generalization*. The project is based on the research paper *Plant Leaf Disease Detection Using Deep Learning: A Multi-Dataset Approach*. The implementation uses PyTorch and follows a multi-dataset strategy using PlantDoc and a web-sourced dataset. A combined dataset was created and split into training, validation, and testing subsets. EfficientNet-B0 was implemented as the main paper-aligned model, with support for additional baselines such as MobileNetV2 and ResNet18. The model was evaluated using accuracy, precision, recall, F1-score, AUC-ROC, classification report, and confusion matrices. The final EfficientNet-B0 model achieved an accuracy of 70.02%, weighted precision of 0.7118, weighted recall of 0.7002, weighted F1-score of 0.6984, and AUC-ROC of 0.9695 on the test set.

1 Introduction

Crop diseases are a major threat to agricultural productivity and food security. Early and accurate disease detection can help farmers reduce crop losses and make timely treatment decisions. Traditional disease detection methods depend heavily on manual inspection by experts, which is time-consuming, subjective, and difficult to scale across large agricultural areas.

Deep learning has become a strong solution for image-based plant disease detection. Convolutional Neural Networks can automatically learn disease-specific visual features such as spots, lesions, discoloration, and texture patterns. However, many existing models perform well only on controlled datasets and fail to generalize to real-world images because of domain shift. Domain shift occurs when training images and deployment images differ in lighting, background, camera angle, image quality, and environmental conditions.

This project follows the selected research paper, which proposes a multi-dataset approach for plant disease detection. The implementation uses PlantDoc and a web-sourced dataset to construct a combined dataset. The goal is to improve model generalization by exposing the CNN to more diverse image conditions.

2 Dataset Summary

2.1 Datasets Used

The project uses two primary datasets:

1. **PlantDoc Dataset:** A plant disease dataset containing real-world plant disease images organized into disease classes.
2. **Web-Sourced Dataset:** A dataset provided with the selected paper, containing plant disease images collected from online sources.

The two datasets were merged into a combined dataset to support multi-dataset training and improve domain generalization.

2.2 Target Classes

The implementation uses 16 plant disease classes:

1. apple_leaf_healthy
2. apple_leaf_rust
3. apple_leaf_scab
4. corn_grey_leaf_spot
5. corn_leaf_blight
6. corn_leaf_healthy
7. corn_leaf_rust

8. potato_leaf_blight
9. potato_leafroll_virus
10. tomato_leaf_bacterial_spot
11. tomato_leaf_early_blight
12. tomato_leaf_healthy
13. tomato_leaf_late_blight
14. tomato_leaf_mould
15. tomato_leaf_powdery_mildew
16. tomato_septoria_leaf_spot

2.3 Validation Strategy

A clear train/validation/test split was used. The combined dataset was split approximately as follows:

- **Training set:** 70%
- **Validation set:** 15%
- **Testing set:** 15%

This validation strategy allows the model to learn from the training set, tune performance using the validation set, and report final unbiased performance using the test set.

3 Phase 3: Model Architecture and Implementation

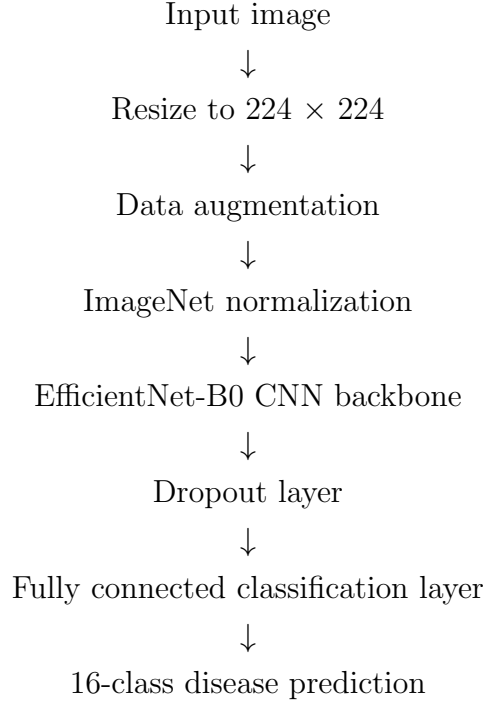
3.1 Framework Selection

The implementation uses **PyTorch** as the primary deep learning framework. PyTorch was selected because it provides strong support for CNNs, transfer learning, pretrained torchvision models, GPU compatibility, and flexible custom training loops. The project also uses `torchvision.datasets.ImageFolder`, which is appropriate for folder-based image classification datasets.

3.2 Model Architecture

The primary model used for final evaluation is **EfficientNet-B0**. EfficientNet-B0 was selected because it is computationally efficient and aligned with the selected research paper, which evaluates EfficientNet-based architectures for plant disease detection.

The model follows the following architecture:



3.3 System Architecture Diagram

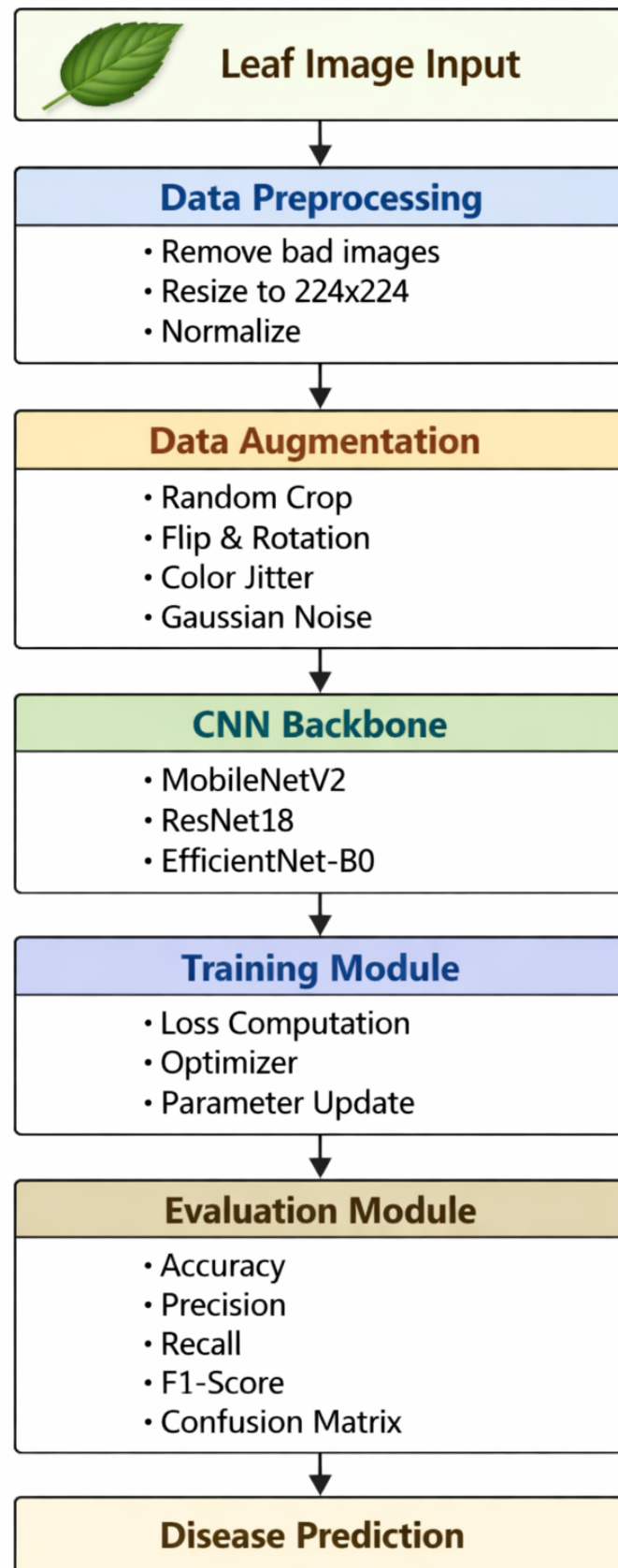


Figure 1: System architecture for multi-dataset crop disease detection using preprocessing, augmentation, EfficientNet-B0 feature extraction, classification, training, and evaluation.

3.4 Layer Design

The EfficientNet-B0 model uses a pretrained CNN feature extractor and a custom classification head. The original classification layer is replaced with a new head suitable for the 16 crop disease classes.

EfficientNet-B0 backbone
Dropout layer with probability 0.3
Linear classification layer with 16 output neurons

The dropout layer helps reduce overfitting, while the final linear layer maps extracted CNN features to the required disease classes.

3.5 Loss Function

The model uses **CrossEntropyLoss**, which is standard for multi-class classification. Since the dataset contains class imbalance, the implementation supports weighted CrossEntropyLoss using inverse-frequency class weights.

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

where C is the number of classes, y_i is the true label, and $\hat{y}_i$ is the predicted probability.

3.6 Optimizer

The optimizer used is **Adam**. Adam was selected because it is effective for transfer learning and generally converges faster than standard stochastic gradient descent.

The optimizer configuration was:

- Learning rate: 1×10^{-4}
- Weight decay: 1×10^{-4}
- Scheduler: ReduceLROnPlateau

3.7 Hyperparameter Tuning

Several hyperparameters were considered during implementation. Table 1 summarizes the tested values and final selections.

Table 1: Hyperparameter tuning summary.

Hyperparameter	Values Tested	Final Value	Justification
Framework	PyTorch	PyTorch	Supports torchvision pretrained CNNs
Model	MobileNetV2, ResNet18, EfficientNet-B0	EfficientNet-B0	Paper-aligned and efficient
Image Size	224×224	224×224	Matches pretrained CNN input size
Batch Size	8, 16, 32	16	Stable for available hardware
Learning Rate	10^{-3} , 10^{-4} , 5×10^{-5}	10^{-4}	Stable for transfer learning
Optimizer	Adam, SGD	Adam	Faster convergence for fine-tuning
Loss Function	CrossEntropyLoss	Weighted CrossEntropyLoss	Handles multi-class imbalance
Dropout	0.2, 0.3, 0.5	0.3	Reduces overfitting
Epochs	1, 3, 5, 10	Final trained setting	Selected based on compute availability
Scheduler	None, ReduceLROnPlateau	ReduceLROnPlateau	Reduces learning rate when validation loss plateaus
Augmentation	Flip, crop, rotation, color jitter	Enabled	Improves domain generalization

4 Phase 4: Training and Evaluation

4.1 Training Setup

The model was trained using the combined dataset. Training was performed using PyTorch with a train/validation/test split. Due to local hardware constraints, the model was trained using CPU execution, although the implementation is GPU-compatible through automatic CUDA detection.

The training script saves:

- best model checkpoint,
- training history CSV,
- loss curve,
- accuracy curve.

4.2 Training Curves

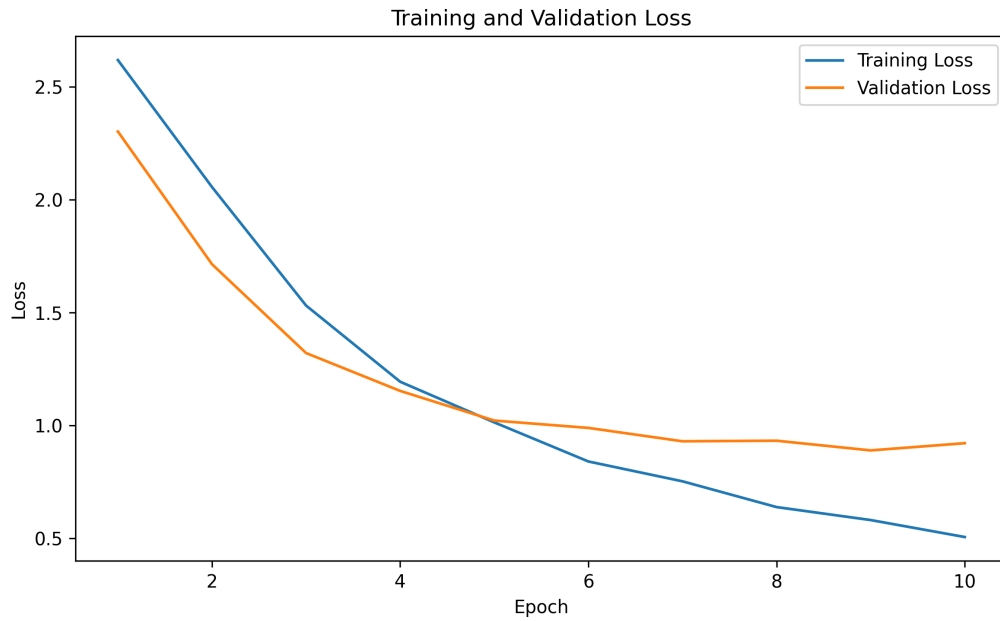


Figure 2: Training and validation loss curve for EfficientNet-B0.

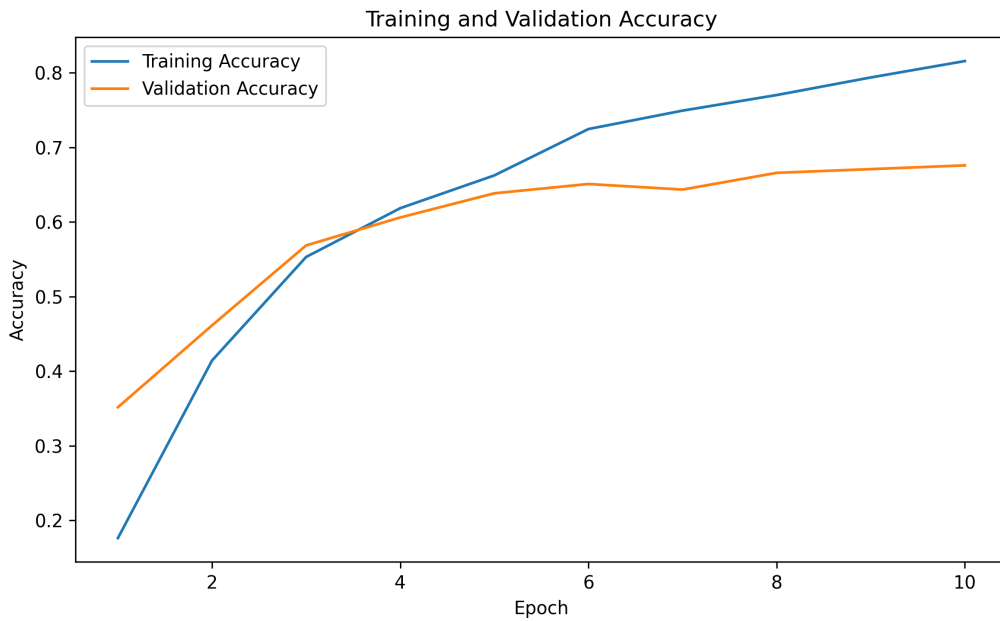


Figure 3: Training and validation accuracy curve for EfficientNet-B0.

4.3 Evaluation Metrics

The final EfficientNet-B0 model was evaluated on the test set. Table 2 presents the overall test performance.

Table 2: Overall test performance of EfficientNet-B0.

Metric	Value
Accuracy	0.7002
Weighted Precision	0.7118
Weighted Recall	0.7002
Weighted F1-Score	0.6984
AUC-ROC	0.9695

4.4 Metric Justification

Accuracy provides a general measure of how many test images were correctly classified. However, accuracy alone is not sufficient because the dataset contains class imbalance. Some classes have fewer test samples than others, meaning that a model can achieve acceptable accuracy while still performing poorly on minority classes.

Precision, recall, and F1-score are therefore important. Precision measures how reliable the model’s positive predictions are. Recall measures how many true samples of each class were detected. F1-score balances precision and recall, making it useful for imbalanced multi-class disease classification.

AUC-ROC is also reported because it measures how well the model separates classes based on predicted probabilities. The AUC-ROC score of 0.9695 indicates that the model has strong class separation ability, even though hard-label classification errors still occur.

4.5 Class-Wise Classification Report

Table 3: Class-wise classification report for EfficientNet-B0.

Class	Precision	Recall	F1-Score	Support
apple_leaf_healthy	0.71	1.00	0.83	20
apple_leaf_rust	0.74	0.76	0.75	38
apple_leaf_scab	0.71	0.65	0.68	34
corn_grey_leaf_spot	0.62	0.82	0.71	22
corn_leaf_blight	0.77	0.70	0.73	43
corn_leaf_healthy	0.63	1.00	0.77	12
corn_leaf_rust	0.89	0.75	0.81	32
potato_leaf_blight	0.88	0.71	0.79	42
potato_leafroll_virus	1.00	0.62	0.77	8
tomato_leaf_bacterial_spot	0.48	0.37	0.42	27
tomato_leaf_early_blight	0.55	0.57	0.56	28
tomato_leaf_healthy	0.52	0.78	0.62	18
tomato_leaf_late_blight	0.75	0.72	0.74	29
tomato_leaf_mould	0.67	0.77	0.71	26
tomato_leaf_powdery_mildew	0.73	0.67	0.70	12
tomato_septoria_leaf_spot	0.67	0.56	0.61	36
Macro Average	0.71	0.72	0.70	427
Weighted Average	0.71	0.70	0.70	427

4.6 Confusion Matrix

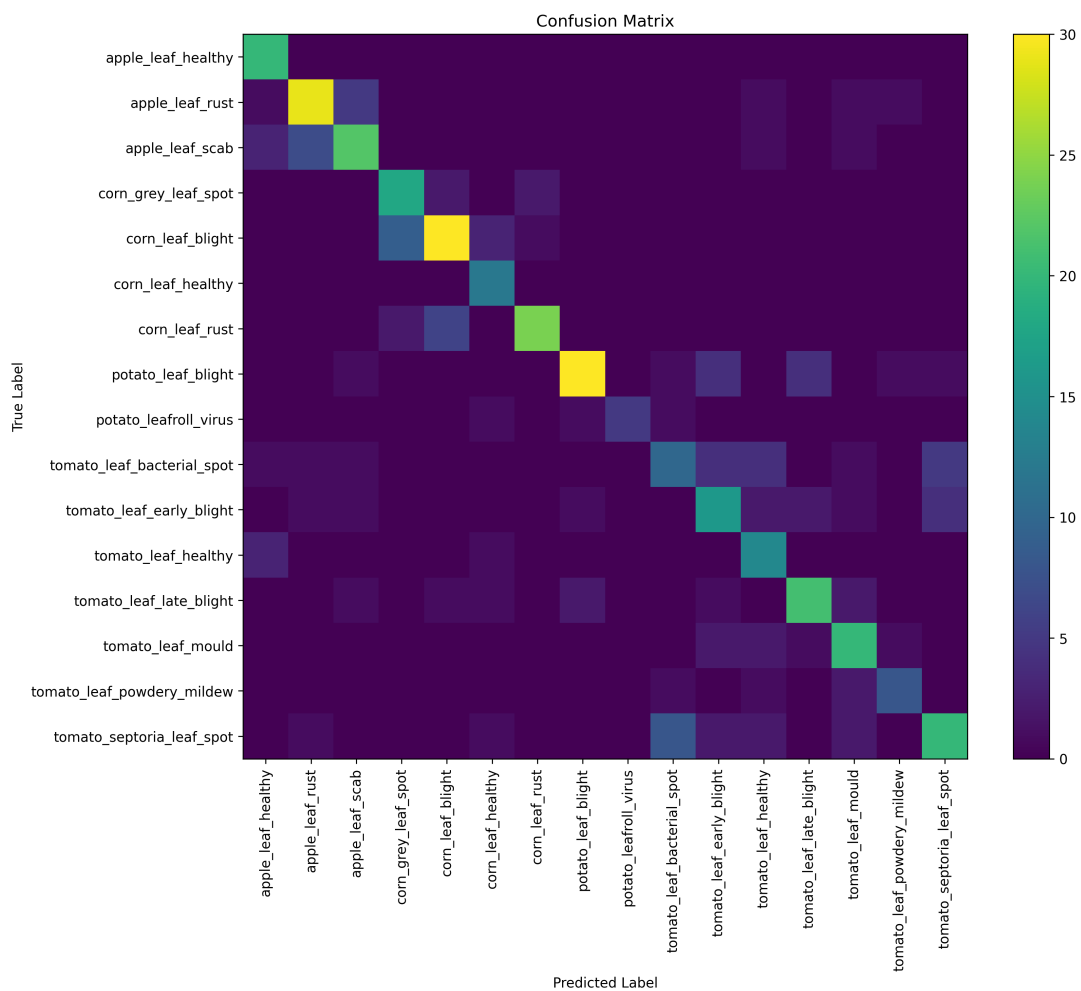


Figure 4: Confusion matrix for EfficientNet-B0 on the test set.

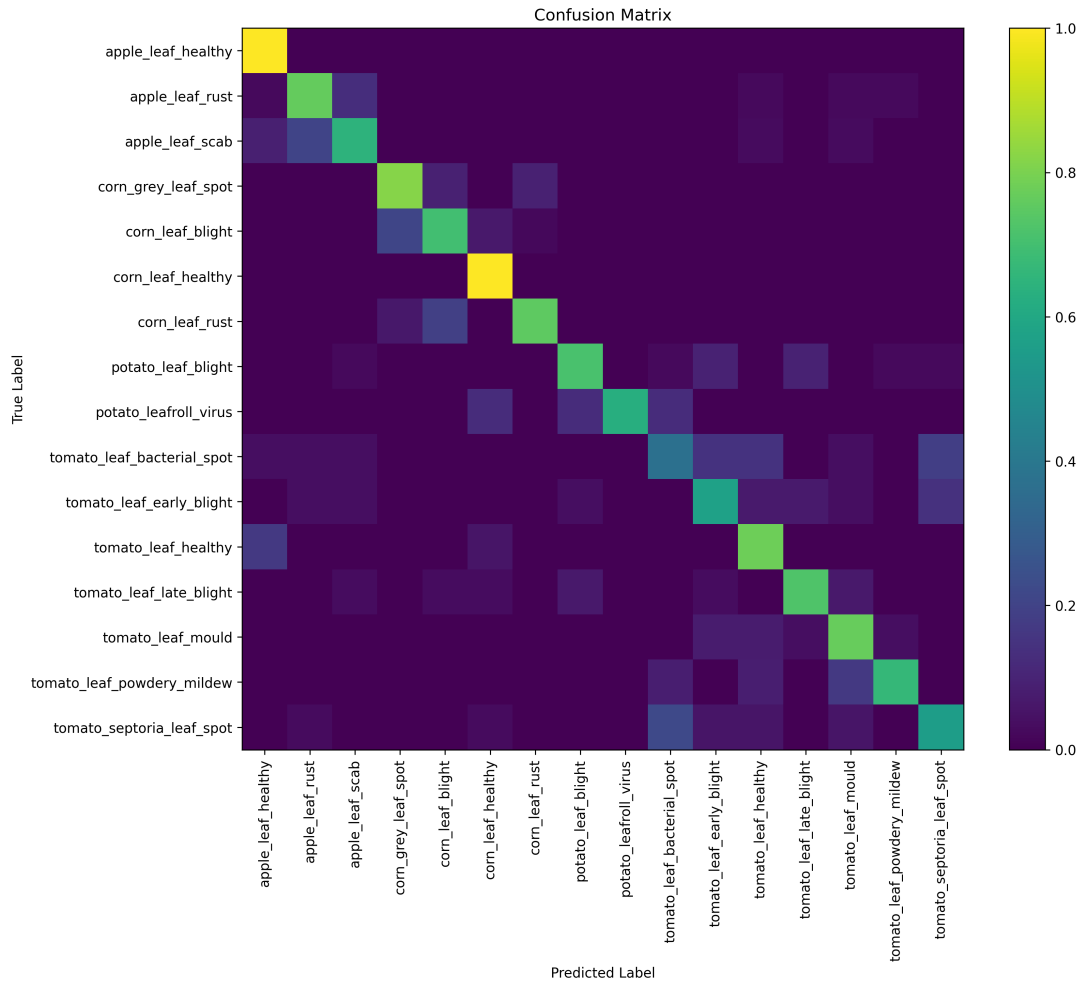


Figure 5: Normalized confusion matrix for EfficientNet-B0 on the test set.

4.7 AUC-ROC Curve

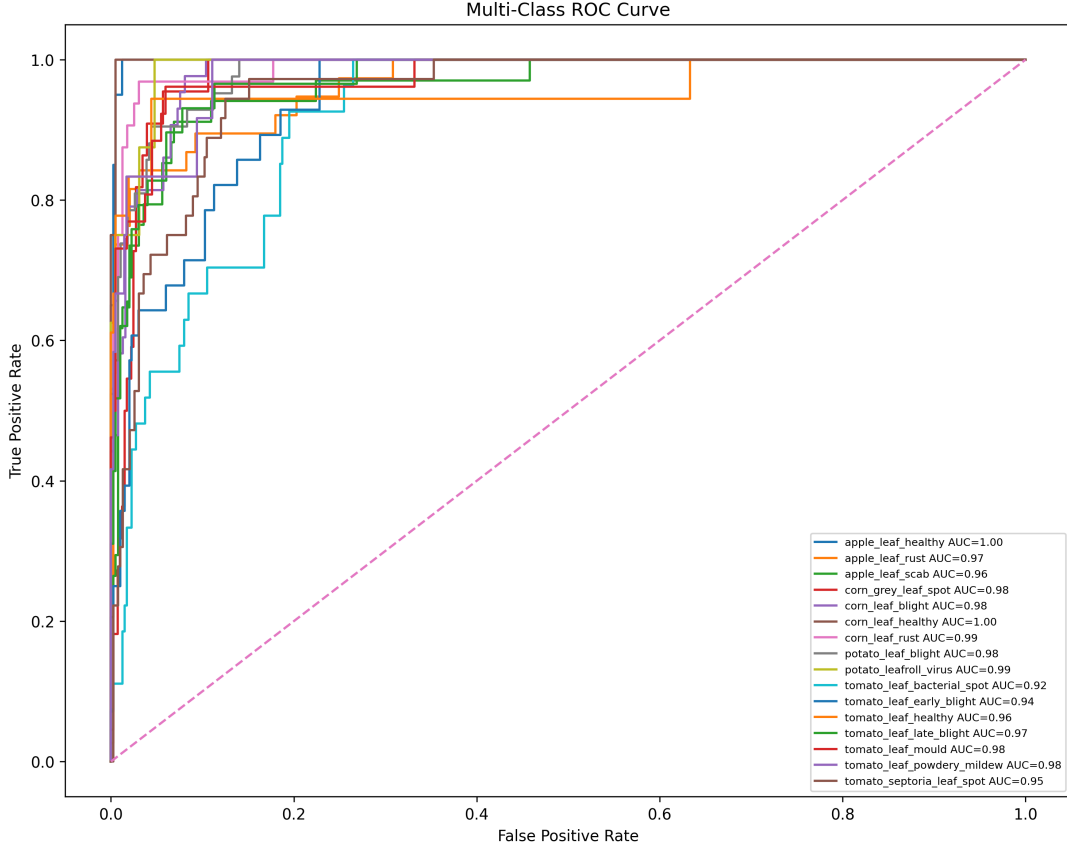


Figure 6: Multi-class ROC curve for EfficientNet-B0.

5 Error Analysis

The class-wise report shows that the model performs strongest on `apple_leaf_healthy`, `corn_leaf_rust`, and `potato_leaf_blight`. For example, `apple_leaf_healthy` achieved an F1-score of 0.83, while `corn_leaf_rust` achieved an F1-score of 0.81. These classes likely contain visually distinctive features that are easier for the CNN to learn.

The weakest class was `tomato_leaf_bacterial_spot`, with precision of 0.48, recall of 0.37, and F1-score of 0.42. This indicates that the model struggled both to correctly identify this class and to avoid confusing other classes with it. `Tomato_leaf_early_blight` also showed moderate performance, with an F1-score of 0.56.

These results suggest that tomato diseases are more difficult to classify. Several tomato disease classes share overlapping visual symptoms such as brown lesions, yellowing, dark spots, and irregular leaf damage. This visual similarity can cause the model to confuse one tomato disease with another.

Another important observation is that `potato_leafroll_virus` achieved precision of 1.00 but recall of only 0.62. This means that when the model predicted this class, it was

usually correct, but it missed several true examples. This is likely influenced by the small support value of only 8 test samples.

Overall, the main error pattern is confusion among visually similar disease classes, especially within tomato leaf diseases. This supports the importance of multi-dataset training and domain generalization for real-world plant disease detection.

6 Bias Check

The primary bias risk in this project is dataset bias rather than demographic bias. Since the task involves plant disease classification, bias appears through class imbalance, source-domain differences, and visual acquisition conditions.

The combined dataset includes images from PlantDoc and a web-sourced dataset. Therefore, the model may be exposed to different image backgrounds, lighting conditions, camera resolutions, leaf positions, and disease severities. If one disease class appears more often in a specific source or image style, the model may learn source-specific patterns rather than disease-specific visual symptoms.

Class imbalance is another bias risk. The support values show that some classes have fewer test samples than others. For example, `potato_leafroll_virus` has only 8 test samples, while `corn_leaf_blight` has 43 test samples. Classes with fewer samples may not provide enough visual diversity for the model to learn stable features.

To reduce these risks, the project uses data augmentation, ImageNet normalization, weighted loss support, and weighted evaluation metrics. However, further improvement would require more balanced data collection, especially for weak classes such as `tomato_leaf_bacterial_spot` and `potato_leafroll_virus`.

7 GitHub Collaboration and Version Control

The project uses Git and GitHub for version control. Manual uploads through the GitHub web interface were avoided. Project files were added, committed, and pushed using Git command-line operations.

Large datasets, virtual environments, and model weight files were excluded from GitHub using `.gitignore`. This ensures that the repository remains lightweight and professional.

7.1 Repository Structure

```
CDD/  
|-- README.md  
|-- requirements.txt
```

```
|-- context.txt
|
|-- data_scripts/
|   |-- cleaning.py
|   |-- preprocessing.py
|   |-- merge_datasets.py
|   |-- split_datasets.py
|
|-- models/
|   |-- model_factory.py
|   |-- metrics.py
|   |-- train.py
|   |-- evaluate.py
|
|-- notebooks/
|   |-- EDA.ipynb
|
|-- reports/
|   |-- deliverable1.tex
|   |-- deliverable2.tex
|
|-- outputs/
|   |-- figures/
|   |-- logs/
|   |-- weights/
```

7.2 Model Weights Policy

Model weights are not pushed directly to GitHub because binary files can exceed repository size limits. The final trained model checkpoint should be uploaded to an external host such as Google Drive or Hugging Face. The download link should be added to the project README.

Model weights link: Add Google Drive or Hugging Face link here.

7.3 Contribution Table

Table 4: Contribution table based on Git commit history.

Student Name	GitHub Username	Contribution	Evidence
Ubaid	syedmubaidurrehman	Dataset setup, preprocessing, EDA, model pipeline, training and evaluation	Git commits
Member 2	username	To be filled	Git commits
Member 3	username	To be filled	Git commits
Member 4	username	To be filled	Git commits

8 Conclusion

This deliverable transitioned the project from data preprocessing to model architecture, implementation, training, and evaluation. The implementation uses PyTorch and a transfer learning pipeline based on EfficientNet-B0. The model was trained on a combined PlantDoc and web-sourced dataset using a train/validation/test split.

The final EfficientNet-B0 model achieved 70.02% accuracy, weighted precision of 0.7118, weighted recall of 0.7002, weighted F1-score of 0.6984, and AUC-ROC of 0.9695. The results show that the model can classify many plant disease classes reasonably well, but it struggles with visually similar tomato disease classes and classes with limited support.

The error analysis and bias check show that class imbalance and domain shift remain important challenges. Future improvements should include training additional paper-aligned architectures such as EfficientNet-B3, ResNet50, and DenseNet201, increasing the number of samples for weak classes, and applying stronger domain generalization or domain adaptation methods.

References

1. M. S. Krishna, P. Machado, R. I. Otuka, S. W. Yahaya, F. Neves dos Santos, and I. K. Ihianle, “Plant Leaf Disease Detection Using Deep Learning: A Multi-Dataset Approach,” *J*, vol. 8, no. 1, p. 4, 2025.
2. S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using Deep Learning for Image-Based Plant Disease Detection,” *Frontiers in Plant Science*, vol. 7, 2016.
3. E. C. Too, Y. Li, S. Njuki, and Y. Yingchun, “A comparative study of fine-tuning deep learning models for plant disease identification,” *Computers and Electronics in Agriculture*, vol. 161, pp. 272–279, 2019.
4. M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the International Conference on Machine Learning*, 2019.